

硕士学位论文

大连理工大学学位论文

XeLaTeX 模板设计与实现

Design and Implementation of XeLaTeX Template for
Dalian University of Technology

学院 (学部): 学院 (学部) 名称
专 业: 专业名称
学 生 姓 名: 作者姓名
学 号: 20200000000
指 导 教 师: 导师姓名 教授
评 阅 教 师: 评阅教师
完 成 日 期: 2024 年 6 月

大连理工大学

Dalian University of Technology

目 录

引 言	1
1 基于 gitgraph 的 git 提交历史可视化	2
1.1 技术架构设计	2
1.1.1 GitGraph.js 核心特性	2
1.1.2 系统架构	2
1.2 核心实现	2
1.2.1 HTML 模板生成	2
1.2.2 样式配置	3
1.2.3 分支管理算法	3
1.2.3.1 主干分支处理	3
1.2.3.2 分支创建与切换	4
1.2.3.3 合并操作处理	4
1.3 功能特性	5
1.3.1 提交信息展示	5
1.3.2 交互功能	5
1.4 性能优化	5
1.4.1 数据处理优化	5
1.4.2 渲染优化	5
1.5 使用示例	5
1.5.1 基本使用	5
1.5.2 输出结果	6
1.6 本章小结	7
2 基于 .git 的提交历史分析与可视化	8
3 目标仓库的代码静态分析与动态分析	9
结 论	10
参考文献	11
附录 A 附录内容名称	12

引 言

1 基于 gitgraph 的 git 提交历史可视化

本章详细介绍如何使用 GitGraph.js 库实现 Git 提交历史的可视化展示。通过分析 Git 仓库的提交数据，生成直观的图形化界面，帮助开发者更好地理解项目的发展历史和分支结构。

1.1 技术架构设计

1.1.1 GitGraph.js 核心特性

GitGraph.js 是一个基于 HTML5 Canvas 的 JavaScript 库，专门用于绘制 Git 图形。该库具备多分支支持能力，能够准确展示复杂的 Git 分支结构，包括分支的创建、合并、分叉等各种操作历史。库提供了丰富的交互式界面功能，支持用户通过鼠标滚轮进行缩放控制，可以拖拽浏览图形的不同部分，让开发者能够从各个角度详细查看提交历史。在样式配置方面，GitGraph.js 提供了高度自定义的选项，允许开发者根据需求调整颜色、布局、字体等各种视觉元素。此外，该库采用响应式设计原则，能够自动适配不同屏幕尺寸，确保在各种设备上都能提供良好的用户体验。

1.1.2 系统架构

整个可视化系统采用清晰的三层架构设计。数据层作为基础层，通过 GitPython 库与 Git 仓库进行深度交互，解析并提取详细的提交信息，包括提交哈希、作者信息、时间戳、父提交关系以及分支标签等核心数据。处理层承担着承上启下的关键角色，Python 脚本不仅对原始 Git 数据进行结构化处理和逻辑分析，还需要将这些数据注入到精心设计的 HTML 模板中，同时集成 GitGraph.js 的配置参数和样式设置。展示层作为最上层，最终通过现代浏览器强大的渲染能力，将抽象的 Git 历史数据转化为直观的可视化图形界面，用户可以清晰地看到分支的创建、合并、分叉等复杂操作的历史轨迹。

1.2 核心实现

1.2.1 HTML 模板生成

主要实现在 `html_generator.py` 文件中，核心函数 `generate_git_tree_html` 负责生成完整的 HTML 页面：

```
def generate_git_tree_html(commits, git_url, output_path="git_tree.html"):
    html_content = f"""
    <!DOCTYPE html>
```

```
<html>
<head>
  <meta charset="UTF-8">
  <title>Git Tree Visualization</title>
  <script src="https://unpkg.com/@gitgraph/js/lib/gitgraph.umd.js">
  </script>
</head>
<body>
  <h1>Git Repository History - {git_url}</h1>
  <div id="graph-container"></div>
</body>
</html>
"""
```

1.2.2 样式配置

为了提升用户体验,系统采用 Metro 风格的视觉设计,在色彩方案方面使用 Material Design 色彩体系,提供了 10 种不同的分支颜色,确保各个分支能够清晰区分。在布局优化方面,系统采用垂直反向布局方式,将最新的提交显示在顶部,这样用户可以直观地看到项目的最新进展。字体设置充分考虑了中英文混合显示的需求,优先使用 Consolas 和微软雅黑字体,既保证了代码的可读性,又确保了中文字符的美观显示。此外,系统还添加了柔和的阴影效果,通过微妙的层次感提升整体视觉体验,让图形界面更加立体和现代化。

1.2.3 分支管理算法

系统实现了智能的分支管理算法,能够准确处理以下场景:

1.2.3.1 主干分支处理

```
const main = gitgraph.branch({
  name: "main",
  style: { label: { display: true } }
});
branches["main"] = main;
branchToLastCommit.set(main, null);
```

1.2.3.2 分支创建与切换

当检测到新的分支引用时，系统会自动创建对应的分支：

```
c.refs.forEach(ref => {
  if (ref.startsWith('refs/heads/') ||
    (ref.startsWith('origin/') && !ref.includes('HEAD')))) {
    const bName = ref.replace('refs/heads/', '')
      .replace('origin/', '');
    if (!branches[bName]) {
      branches[bName] = targetBranch.branch({
        name: bName,
        style: { label: { display: true } }
      });
    }
  }
});
```

1.2.3.3 合并操作处理

对于合并提交（包含多个父提交），系统会智能识别并执行合并操作：

```
if (c.parents.length > 1) {
  for (let i = 1; i < c.parents.length; i++) {
    const otherBranch = commitToBranch[c.parents[i]];
    if (otherBranch && otherBranch !== targetBranch) {
      targetBranch.merge({
        branch: otherBranch,
        commitOptions: commitOptions
      });
      break;
    }
  }
}
```

1.3 功能特性

1.3.1 提交信息展示

每个提交节点都包含丰富的信息展示，在提交哈希方面显示完整的 SHA-1 哈希值，方便开发者进行精确的定位和查找。提交信息部分则显示详细的提交说明文字，让开发者能够快速了解每次提交的具体内容和目的。作者信息包含了作者姓名和提交时间，为团队协作提供了必要的时间线和责任人信息。在视觉标识方面，系统采用圆点表示提交节点，并通过不同颜色区分各个分支，使得复杂的 Git 历史结构一目了然。

1.3.2 交互功能

系统提供了丰富的交互功能，在缩放控制方面支持鼠标滚轮缩放图形，用户可以根据需要放大查看细节或缩小查看整体结构。拖拽浏览功能允许用户拖拽查看图形的不同部分，即使对于包含大量提交历史的项目也能轻松导航。分支标签功能清晰显示各分支名称，帮助用户快速识别和定位不同的开发线路。响应式适配机制确保图形能够自动适应容器大小，无论是在大型显示器还是小型设备上都能够提供最佳的视觉效果。

1.4 性能优化

1.4.1 数据处理优化

在数据处理方面，系统采用批量处理策略，一次性处理所有提交数据，避免了重复计算带来的性能损耗。内存管理方面使用 Map 数据结构来优化查找性能，确保在处理大量提交数据时仍能保持高效的查询速度。引用缓存机制则通过缓存分支和提交的映射关系，减少了重复的查找操作，显著提升了整体处理效率。

1.4.2 渲染优化

渲染优化方面，系统充分利用 HTML5 Canvas 技术提供硬件加速，确保图形渲染的流畅性和高性能。布局算法经过精心优化，能够有效避免分支交叉和重叠问题，即使在复杂的 Git 历史结构中也能保持清晰的视觉效果。样式预编译技术则通过提前编译 CSS 样式，减少了运行时的计算开销，进一步提升了渲染速度和用户体验。

1.5 使用示例

1.5.1 基本使用

```
commits = [  
    {
```

```

    "hash": "abc123",
    "hashFull": "abc123def456...",
    "message": "feat: 添加新功能",
    "author": "张三",
    "date": "2024-01-01",
    "parents": [],
    "refs": ["HEAD", "refs/heads/main"]
  }
]

generate_git_tree_html(commits, "https://github.com/user/repo")

```

1.5.2 输出结果

生成的 HTML 文件包含完整的 Git 图形可视化，用户可以在浏览器中直接打开查看，无需额外的配置或依赖。该文件可以作为静态文件保存，供后续重复使用或分享给团队成员。同时，生成的 HTML 代码也可以轻松嵌入到其他网页中，作为项目文档或报告的一部分。此外，系统还支持打印和截图功能，方便用户将可视化结果保存为图片或 PDF 格式，用于演示文稿或文档编制。图1.1展示了生成结果的可视化效果。



图 1.1 Git 提交历史可视化效果图

1.6 本章小结

本章详细介绍了基于 GitGraph.js 的 Git 提交历史可视化实现。通过 Python 后端处理和前端图形渲染的结合，提供了一个功能完整、性能优良的 Git 历史可视化解决方案。该系统不仅能够准确展示复杂的 Git 分支结构，还提供了丰富的交互功能和优雅的视觉体验。

2 基于.git 的提交历史分析与可视化

3 目标仓库的代码静态分析与动态分析

结 论

结论是理论分析和实验结果的逻辑发展，是整篇论文的归宿。结论是在理论分析、试验结果的基础上，经过分析、推理、判断、归纳的过程而形成的总观点。结论必须完整、准确、鲜明、并突出与前人不同的新见解。

参考文献

- [1] K. Reckdahl 原著王磊 译. Using Import graphics in $\text{\LaTeX}2\epsilon$, $\text{\LaTeX}2\epsilon$ 插图指南[A]. 2000.
- [2] 王仁宏, 施锡泉, 罗钟铉, 等. 多元样条函数及其应用[M]. 北京: 科学出版社, 1994.
- [3] 吴凯. GBT7714-2005.bst: 利用 Bib $\text{\TeX}$ 生成符合 GB/T 7714-2005 的参考文献[A]. 1.0 β 3. 1.0 β 3, 2006.

附录 A 附录内容名称